

Future

- L'interfaccia `Future<V>` permette di rappresentare un risultato (di tipo V) che sarà disponibile in un momento successivo (futuro). Rappresenta un **compito** che esegue in modo **asincrono** ed è un riferimento al risultato

- L'avvio di un'attività per mezzo di un `Future` permette al thread chiamante di continuare a fare del lavoro, anziché aspettare il risultato dell'operazione

```
// prima di Java 8
ExecutorService esecutore = Executors.newCachedThreadPool();
Future<Double> risultato = esecutore.submit(new Callable<Double>() {
    public Double call() {
        return calcola();
    }
});
calcolaAltro();
```

NUOVO THREAD

- `submit()` avvia un'operazione che ritorna un `Future` che rappresenta il risultato dell'operazione. La chiamata ritorna subito *non si blocca*
- `Executors` e `newCachedThreadPool()` permettono di creare un nuovo thread, ovvero l'istanza di `ExecutorService`. Tale istanza esegue l'operazione richiesta (questa operazione, supponiamo, prende tanto tempo per eseguire)
- `ExecutorService` è un'interfaccia che fornisce il metodo `submit()`, che prende in ingresso un `Callable`. Un `Callable` è un'interfaccia che dichiara il metodo `call()` che ritorna un risultato

1

Prof. Tramontana - Dicembre 2024

Considerazioni Su Future

- `Future` **non** permette di implementare codice conciso quando
 - Si vuol fornire il risultato sebbene il **compito avviato non si è ancora concluso** [vedi dopo, versione alternativa con `complete()`]
 - Si vuol usare il risultato della prima esecuzione asincrona in un'altra **esecuzione asincrona** [versione alternativa: vedi dopo `thenApply()`]
 - Si vuol aspettare il completamento di tutti i compiti eseguiti in modo **asincrono** [versione alternativa: vedi dopo `allOf()`]
 - Si vuol aspettare il completamento solo del più veloce fra i compiti eseguiti in modo **asincrono** e si vuol prendere il risultato [alternativa: vedi `anyOf()`]
 - Si vuole la notifica del completamento di una attività, con una **callback**, così da poter avviare altre attività col risultato, non appena disponibile. Ovvero, reagire al completamento di un `Future` [alternativa vedi `callback`]

3

Prof. Tramontana - Dicembre 2024

Lettura Risultato Da Future: `get()`

- Il thread aggiuntivo è fornito da `ExecutorService`
- Quando non si può più procedere senza il risultato dell'operazione che prende tanto tempo e che sta eseguendo sul thread aggiuntivo, si può prendere il risultato chiamando `get()` sul `Future` (ovvero sulla variabile risultato)
- `get()` restituirà immediatamente il valore se l'operazione era già stata completata, altrimenti il thread che chiama `get()` si **blocca** in attesa che il risultato diventi disponibile
- Il metodo `get()` con un parametro (overloaded) accetta un **timeout** che definisce il tempo massimo di attesa per il thread che aspetta il risultato. Se si raggiunge il tempo massimo di attesa si ha l'eccezione `TimeoutException`
- Il metodo `isDone()` ritorna un `boolean` che è `true` se l'esecuzione è stata completata

2

Prof. Tramontana - Dicembre 2024

CompletableFuture `supplyAsync()`

- Da Java 8 è disponibile `CompletableFuture`, una classe che implementa `Future` e `CompletionStage`
 - `CompletableFuture` permette di avviare **esecuzioni parallele in modo conciso**
 - Sia `computePrice()` un metodo che, accedendo a un servizio remoto, ha un **tempo di esecuzione lungo**, `Double computePrice(String s)`
 - Per **avviare un thread separato che esegue `computePrice()`**
- ```
var getPrice = CompletableFuture.supplyAsync(() -> p.computePrice("pen"));
```
- `supplyAsync()` è un metodo **static** che prende in ingresso un `Supplier<U>` (interfaccia funzionale avente il metodo `get()` che restituisce un valore di tipo U)
  - `supplyAsync()` avvia un thread per eseguire il metodo `get()` del `Supplier` passato, e restituisce un nuovo `CompletableFuture` parametrizzato con U
  - Il `Supplier` verrà eseguito da uno dei thread degli `Executors`

4

Prof. Tramontana - Dicembre 2024

## CompletableFuture get()

- `get()` permette di estrarre il valore dal `CompletableFuture`
- `get()` aspetta, se necessario, il completamento dell'operazione
- `get()` può lanciare eccezioni, quando: il thread è stato interrotto, l'esecuzione del thread è stata cancellata, o l'esecuzione ha generato un'eccezione

```
var getPrice = CompletableFuture.supplyAsync(() -> p.computePrice("pen"));

try {
 System.out.println("Caller: price " + getPrice.get());
} catch (InterruptedException e) {
 e.printStackTrace();
} catch (ExecutionException e) {
 e.printStackTrace();
}
```

5

Prof. Tramontana - Dicembre 2024

## CompletableFuture complete()

- `isDone()` ritorna true se l'esecuzione è stata completata in uno dei possibili modi (normale, con eccezioni, cancellata)
- boolean `complete(T value)` se non ha già completato l'esecuzione imposta il valore restituito poi da `get()` a `value`
- Ritorna true se il valore è stato impostato, quindi lo stato è passato a completato per via della chiamata a `complete()`, altrimenti ritorna false

```
var getPrice = CompletableFuture.supplyAsync(() -> p.computePrice("book"));
boolean fixed = getPrice.complete(20d);
if (fixed) System.out.println("Yes, fixed");
```

6

Prof. Tramontana - Dicembre 2024

## CompletableFuture con Timeout

- `get()` permette di estrarre il valore da un `CompletableFuture`
- La variante con timeout di `get()` permette di aspettare un numero massimo di `TimeUnit` il completamento dell'operazione
- `get(long timeout, TimeUnit unit)`
- La variante con timeout di `complete()` aspetta un certo tempo prima di completare con il dato valore, ritorna questo `CompletableFuture`
- `completeOnTimeout(T value, long timeout, TimeUnit unit)`

```
getPrice.get(50L, TimeUnit.MILLISECONDS);
```

```
getPrice.completeOnTimeout(20d, 50L, TimeUnit.MILLISECONDS);
```

- Il metodo `orTimeout()` aspetta un certo tempo l'esecuzione del thread e quindi completa con un'eccezione se il thread non ha completato prima, ritorna questo `CompletableFuture`

```
getPrice.orTimeout(50L, TimeUnit.MILLISECONDS);
```

7

Prof. Tramontana - Dicembre 2024

## CompletableFuture thenApply()

- Si vuol usare il risultato di una prima esecuzione asincrona in un'altra esecuzione asincrona. La chiamata avviene in modo sincrono, ovvero non appena il risultato della prima esecuzione è disponibile
- Sia abbiano i metodi double `computePrice(String s)` e `String transform(double d)`, si vuol eseguire il primo metodo in modo asincrono e iniziare a eseguire il secondo metodo appena il primo ha completato l'esecuzione
- `thenApply()` prende in ingresso una funzione che usa il contenuto del `CompletableFuture getPrice`
- Quando il risultato di `computePrice()` sarà disponibile, `transform()` sarà chiamato (e userà il risultato, che viene automaticamente estratto dal `CompletableFuture`). `thenApply()` non blocca il codice chiamante
- Inoltre, in modo asincrono, il risultato di due operazioni asincrone (`computePrice()` e `transform()`) verrà inserito in un unico `CompletableFuture getStrPrice`

```
var getPrice = CompletableFuture.supplyAsync(() -> p.computePrice("book"));
var getStrPrice = getPrice.thenApply(d -> p.transform(d));
```

Prof. Tramontana - Dicembre 2024

## CompletableFuture allOf()

- Si vuol aspettare il completamento di tutti i compiti svolti da un certo numero di esecuzioni asincrone. Primo passo: si avviano le esecuzioni asincrone con il metodo statico `supplyAsync()` e si mette il risultato su un elemento di un array di `CompletableFuture`
- Quindi, si usa il metodo statico `allOf()` passando l'array che ritorna un `CompletableFuture<Void>` che è completato quando tutti gli elementi dell'array hanno completato (`allOf()` può prendere vari parametri anziché un array)
- Infine, si può usare `join()` per aspettare che tutte le esecuzioni abbiano completato, quest'ultimo ritorna un `Void`

```
CompletableFuture<Double>[] prezzi = new CompletableFuture[2];
prezzi[0] = CompletableFuture.supplyAsync(() -> calcolaPrezzo("prodotto0"));
prezzi[1] = CompletableFuture.supplyAsync(() -> calcolaPrezzo("prodotto1"));
CompletableFuture.allOf(prezzi).join();
try {
 System.out.println(prezzi[0].get());
 System.out.println(prezzi[1].get());
} catch (InterruptedException e) {
 e.printStackTrace();
} catch (ExecutionException e) {
 e.printStackTrace();
}
```

9

Prof. Tramontana - Dicembre 2024

## CompletableFuture anyOf()

- Si vuol aspettare il completamento solo del più veloce fra i compiti svolti in modo asincrono e si vuol prendere il risultato
- Si usa il metodo statico `anyOf()` per avere un `CompletableFuture p` che è completato quando uno degli elementi dell'array ha completato. Il `CompletableFuture p` contiene il risultato di chi ha completato

```
CompletableFuture<Double>[] prezzi = new CompletableFuture[2];
prezzi[0] = CompletableFuture.supplyAsync(() -> calcolaPrezzo("prodotto0"));
prezzi[1] = CompletableFuture.supplyAsync(() -> calcolaPrezzo("prodotto1"));
var p = CompletableFuture.anyOf(prezzi);
```

- Il tipo `CompletableFuture` ritornato da `anyOf()` contiene un `Object`
- Se un compito completa con una eccezione, il `CompletableFuture p` completa anch'esso con una eccezione

10

Prof. Tramontana - Dicembre 2024

## Callback thenApply() e thenAccept()

- Sono disponibili vari metodi di `CompletableFuture` per permettere di registrare una callback, ovvero una chiamata quando si verifica un evento
- Registrazione callback per mezzo di `thenApply()`
  - In un esempio precedente, il metodo `transform()` è stato registrato con `thenApply()` ed è chiamato quando l'esecuzione precedente è completa, e il valore di ritorno (prezzo) diventa disponibile
- Registrazione callback per mezzo di `thenAccept()`
  - Il metodo `thenAccept()` prende come parametro un `Consumer`, ovvero un metodo che consuma il valore del `CompletableFuture`  
`prSconto.thenAccept(System.out::println);`
  - Il metodo `thenAccept()` restituisce un `CompletableFuture<Void>`
  - A differenza di `thenApply()`, `thenAccept()` restituisce un `CompletableFuture` che può solo indicare se il compito è completo o meno, e non può tenere un risultato, poiché `thenAccept()` prende come parametro un `Consumer`

11

Prof. Tramontana - Dicembre 2024

## Conclusione Con Eccezione

- Il metodo `completeExceptionally()` fa sì che l'invocazione di `get()` lanci un'eccezione se l'esecuzione rappresentata dal `CompletableFuture` su cui è invocato non è stata già conclusa

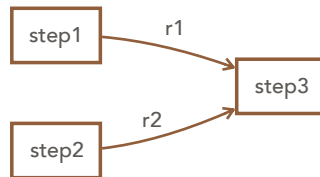
```
CompletableFuture<String> cf = CompletableFuture.supplyAsync(() -> calc());
cf.completeExceptionally(new RuntimeException("exceptionally"));
try {
 System.out.println("value: " + cf.get());
} catch (InterruptedException | ExecutionException e) {
 System.out.println("excep:" + e.getMessage());
}
```

12

Prof. Tramontana - Dicembre 2024

## Alcuni Step Di Esecuzione Asincrona

- Spesso occorre eseguire più passi (step) di esecuzione per completare un lavoro (task o workflow)
- Supponiamo che due step indipendenti step1 e step2 siano avviati ciascuno in un thread dedicato, ciascuno con il proprio tempo d'esecuzione
- I risultati prodotti (r1 e r2) da ciascuno step sono utili ad uno step successivo step3 che quindi deve aspettare l'esecuzione degli step precedenti per poter eseguire



13

Prof. Tramontana - Dicembre 2024

## Metodi Per Non Bloccare Il Chiamante

- `thenCombine()` è un metodo di `CompletableFuture` che prende due parametri: (i) un `CompletableFuture fsc`, e (ii) una funzione con due argomenti; quindi restituisce un `CompletableFuture`
- Quando il `CompletableFuture fpr` su cui è chiamato `thenCombine()` e quello passato come parametro `fsc` hanno completato, allora i due risultati sono passati come argomento alla funzione fornita come secondo parametro; `thenCombine()` restituisce un `CompletableFuture` con il risultato della funzione passata

```
var fsc = CompletableFuture.supplyAsync(() -> stepSconto());
var fpr = CompletableFuture.supplyAsync(() -> stepPrezzo());
var fpp = fpr.thenCombine(fsc, (a, b) -> a * b);
```

- `thenCompose()` prende una funzione con un parametro, e restituisce un `CompletableFuture`
- Quando l'esecuzione sul `CompletableFuture fpp` su cui è chiamato `thenCompose()` è completata, chiama la funzione passata e restituisce il `CompletableFuture` che è il risultato della funzione chiamata

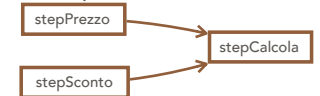
```
var fps = fpp.thenCompose(p -> stepCalcola(p));
```

15

Prof. Tramontana - Dicembre 2024

## Esempio Di Esecuzione Di Step

- Siano `stepPrezzo()` e `stepSconto()` due operazioni che possono essere avviate in parallelo (ovvero step1 e step2) e restituiscono ciascuno un `double`
- Sia `stepCalcola()` un'operazione (step3) che prende un parametro `double` e restituisce un `CompletableFuture<Double>`



- Soluzione 1

```
var fsc = CompletableFuture.supplyAsync(() -> stepSconto());
var fpr = CompletableFuture.supplyAsync(() -> stepPrezzo());
// blocking wait
var d = stepCalcola(fpr.join() * fsc.join());
System.out.println("prezzo scontato: " + d.join());
```

- `fsc` e `fpr` sono avviati e restituiti in modo asincrono
- La chiamata a `stepCalcola()` blocca il chiamante poiché l'argomento passato deve aspettare il completamento di entrambe le operazioni `stepSconto()` e `stepPrezzo()`
- Il metodo `join()` di `CompletableFuture` restituisce il valore del metodo eseguito quando l'esecuzione corrispondente è stata completata

14

Prof. Tramontana - Dicembre 2024

## Esecuzione Di Step

- Soluzione 2

```
var fsc = CompletableFuture.supplyAsync(() -> stepSconto());
var fpr = CompletableFuture.supplyAsync(() -> stepPrezzo());
var fpp = fpr.thenCombine(fsc, (a, b) -> a * b);
var fps = fpp.thenCompose(p -> stepCalcola(p));
System.out.println("prezzo scontato: " + fps.join());
```

oppure

```
var fsc = CompletableFuture.supplyAsync(() -> stepSconto());
var fpr = CompletableFuture.supplyAsync(() -> stepPrezzo())
 .thenCombine(fsc, (a, b) -> a * b)
 .thenCompose(p -> stepCalcola(p));
System.out.println("prezzo scontato: " + fps.join());
```

- Si eseguono gli step preliminari (`stepSconto()` e `stepPrezzo()`) in modo asincrono, e si indica che lo `stepCalcola()` dipende dai risultati dei precedenti step (`fsc` e `fps`), senza che il chiamante venga bloccato

16

Prof. Tramontana - Dicembre 2024

# Confronto Fra Metodi

- Si noti che
  - `stepCalcola()` restituisce un `CompletableFuture<Double>`
  - La chiamata a `thenCompose()` avvia l'operazione `stepCalcola()` e restituisce un valore di ritorno di tipo `CompletableFuture<Double>`
  - Se usassimo `thenApply()` anziché `thenCompose()`, avremmo un annidamento, ovvero un valore di ritorno di tipo `CompletableFuture<CompletableFuture<Double>>`